

QuantumVault

Mid-Review Technical Progress Report

Week 1 & Week 2 Implementation

Project	QuantumVault
Review Scope	Week 1 – Week 2
Primary Branch	Krunal-Jadav/QuantumVault
Core Technologies	Rust, FUSE, C, liboqs, ML-KEM-768, AES-256-GCM
Review Status	Week 1 and Week 2 implementation completed

Purpose: This report summarizes the QuantumVault implementation, testing, cryptographic integration, and validation work completed during the first two project weeks for the mid-review.

1. Project Overview

QuantumVault is a Rust-based userspace filesystem using FUSE. The project combines filesystem operations with post-quantum cryptography and authenticated symmetric encryption. The Week 1 foundation established a working FUSE filesystem, while Week 2 integrated ML-KEM-768 through C/liboqs FFI and added AES-256-GCM encrypted backing-file storage.

2. Week 1 — FUSE Filesystem Foundation

Area	Completed Work
Rust/FUSE Setup	Initialized the Rust FUSE project using the fuser crate and configured the build system.
Filesystem	Implemented a minimal QuantumVault filesystem with a root directory and hello.txt.
Filesystem Operations	Implemented lookup, getattr, readdir, and read operations.
Mounting	Created and tested a FUSE mount point at /tmp/quantumvault-mnt during the initial implementation.
Build Validation	Verified successful cargo build and generated the QuantumVault FUSE binary.
Testing	Validated filesystem visibility and hello.txt read behavior.

3. Week 2 — Cryptography and Secure Storage

Component	Implementation / Validation
C/liboqs FFI	Integrated Rust with the C cryptographic layer using an FFI function quantumvault_ffi_test().
ML-KEM-768	Initialized ML-KEM-768 and verified public key, secret key, ciphertext, and shared-secret sizes.
KEM Cycle	Implemented keypair generation, encapsulation, decapsulation, and shared-secret comparison.
PQC Validation	Verified successful ML-KEM-768 cycle with 'Shared secrets match!' and Rust FFI success.
AES-256-GCM	Added the aes-gcm Rust crate and implemented authenticated encryption/decryption.
Encrypted Backing File	Stored encrypted file data in /tmp/quantumvault_encrypted.bin instead of plaintext.
FUSE Write	Implemented encrypted write handling and verified plaintext-to-ciphertext conversion.
FUSE Read	Implemented decryption on read and verified recovery of the original plaintext.
Tamper Detection	Modified ciphertext and verified AES-GCM authentication/decryption failure.
Recovery	Restored the original ciphertext backup and verified successful plaintext recovery.

4. Technical Architecture Implemented

Layer	Role
FUSE / Linux	Provides the userspace filesystem mount and filesystem interface.
Rust	Implements filesystem logic, FUSE callbacks, encryption/decryption flow, and application control.
C FFI Layer	Provides the interface between Rust and liboqs.
liboqs	Provides ML-KEM-768 post-quantum key encapsulation functionality.
AES-256-GCM	Provides authenticated encryption for filesystem data stored in the backing file.
Encrypted Backing File	Stores ciphertext rather than plaintext data.

5. Crypto Audit Evidence

Evidence	Result
ML-KEM-768 keypair generation	PASS
ML-KEM-768 encapsulation	PASS
ML-KEM-768 decapsulation	PASS
Shared-secret equality	PASS
AES-GCM encryption	PASS
Ciphertext stored at rest	PASS
AES-GCM decryption	PASS
Ciphertext tamper detection	PASS
Post-tamper recovery	PASS

6. Evidence / Screenshot Index

ID	Suggested Evidence File
01	01_ml_kem_768_key_encapsulation_success.png
02	02_aes_gcm_encryption_success.png
03	03_aes_gcm_ciphertext_at_rest.png
04	04_aes_gcm_decryption_success.png
05A	05a_terminal_aes_gcm_auth_failure.png
05B	05b_cat_tampered_file_failure.png
06	06_aes_gcm_recovery_after_tamper.png

7. Tools and Technologies

Tool / Technology	Purpose
Kali Linux	Development and testing environment.
Rust / Cargo	Filesystem implementation and dependency/build management.
FUSE / fuser	Userspace filesystem implementation.
C	Cryptographic FFI layer.
liboqs	Post-quantum cryptographic implementation.
ML-KEM-768	Post-quantum key encapsulation mechanism.
AES-256-GCM	Authenticated symmetric encryption.
Git / GitHub	Version control, branch management, commits, and remote backup.
OpenSSL / xxd	Supporting cryptographic and binary-data inspection during audit.

8. Git / Development Progress

Commit	Description
QV-01	Initialize repository with project documentation and README.
QV-02	Implement ML-KEM-768 demonstration.
QV-03	Initialize Rust FUSE project.
QV-04	Implement Week 1 FUSE filesystem.
QV-05	Add project gitignore.
QV-06	Restructure filesystem for Week 2.
QV-07	Restructure crypto for Week 2.
QV-08	Integrate Rust C FFI with liboqs.
QV-09	Implement ML-KEM-768 key encapsulation cycle.
QV-10	Week 2 AES-GCM encrypted file-storage work / current implementation.

9. Current Status

At the end of Week 2, the QuantumVault prototype has a working FUSE filesystem, successful ML-KEM-768 execution through the Rust/C/liboqs boundary, and an AES-256-GCM encrypted backing-file workflow. The implementation has been validated through functional tests and a ciphertext tamper test demonstrating authenticated decryption failure.

10. Next Phase — Week 3

Week 3 can build on the validated Week 1–2 foundation. Recommended focus areas include improving key management, removing hard-coded cryptographic material, strengthening file handling and metadata behavior,

improving error handling, adding more complete filesystem operations, and preparing the final technical documentation and demonstration.

11. Mid-Review Conclusion

Week 1: FUSE filesystem foundation completed and tested.

Week 2: Post-quantum ML-KEM-768 integration, C/Rust FFI, AES-256-GCM encrypted storage, decryption, ciphertext inspection, tamper detection, and recovery validation completed.

Overall: The project has progressed from a basic userspace filesystem prototype to a cryptography-enabled encrypted storage prototype suitable for the mid-review stage.